Begleitdokumentation zur Bachelorarbeit · Stand: 28. Oktober 2026

MIPS-Emulator

Softwaredokumentation – vom Lern-Emulator (2018) zum Referenzmodell für den cristian1604-MIPS mit Bus-angebundenem Beschleuniger für die Multiplikation komplexer Zahlen

Original 2018 – händische Implementierung **Erweiterung 2026 – KI-gestützt** **gemischt**

Diese Farbkennzeichnung wird im gesamten Dokument verwendet – in Überschriften, Tabellen und Grafiken. Blau steht für Code aus der ursprünglichen Implementierung von 2018, Orange für die Erweiterung von 2026.

Inhalt

1. Abgrenzung: Originalimplementierung und Erweiterung
2. Systemüberblick
3. Die Originalimplementierung von 2018
4. Zielplattform cristian1604/mips
5. Verarbeitungskette der Erweiterung
6. Assembler
7. CPU-Emulation
8. Bus und Speicherbelegung
9. Beschleuniger für komplexe Multiplikation
10. Hex-Export
11. Klassenstruktur
12. Bedienung
13. Beispielprogramme und Ergebnisse
14. Test und Verifikation
15. Grenzen und Ausblick
16. Anhang: Dateiverzeichnis mit Herkunft

1 Abgrenzung: Originalimplementierung und Erweiterung

Das Projekt besteht aus zwei klar unterscheidbaren Entwicklungsständen. Diese Abgrenzung steht bewusst am Anfang, damit bei jeder Komponente nachvollziehbar ist, von wem und wann sie stammt.

Originalimplementierung 2018

- Händisch vom Autor geschrieben, im Jahr 2018
- Ziel: einfacher Lern-Emulator für MIPS-Assembler (Vorlesung, Tutorium, Nachhilfe)
- Ins Git-Repository eingecheckt in Commit `874bd1e` („CPU geschrieben, loader implementiert, kleine programme laufen“, Commit-Datum 03.10.2025) und `9b2462d` (README)
- Umfang: 395 Zeilen Java, 62 Zeilen Assembler

Erweiterung Oktober 2026

- Erstellt mit dem KI-Assistenten **Claude** (Anthropic) in der Entwicklungsumgebung Claude Code
- Vorgaben des Autors: cristian1604/mips als Ziel-CPU, Anbindung des Beschleunigers über einen Bus, Hex-Ausgabe nach erfolgreicher Simulation, Dokumentation aller Änderungen
- Ziel: Referenzmodell für die Hardware-Entwicklung in der Bachelorarbeit
- Umfang: 1 736 Zeilen Java (davon 229 in bestehenden Dateien), 133 Zeilen Assembler

Java-Zeilen Original (erhalten)

343

von 395 Zeilen aus 2018

Java-Zeilen Erweiterung

1 736

1 507 in neuen Dateien

Unverändert übernommene Dateien

3

CPU, simpleCPU, specialproposeReg

Neue Dateien

19

16 Java, 3 Assembler

Wie die Abgrenzung ermittelt wurde

Grundlage ist die Git-Historie. Als Originalstand gilt der Inhalt von Commit `9b2462d` (HEAD vor Beginn der Erweiterung). Alles, was im Arbeitsverzeichnis gegenüber diesem Stand neu hinzugekommen oder geändert ist, gehört zur Erweiterung. Bei geänderten Dateien wurden die Zeilen mit `git diff --numstat` ausgezählt: „erhalten“ sind Originalzeilen, die unverändert geblieben sind, „ergänzt“ sind neu hinzugekommene Zeilen.

Hinweis: Das Commit-Datum (2025) weicht vom Entstehungsjahr (2018) ab, weil der Code erst nachträglich in das Repository aufgenommen wurde.

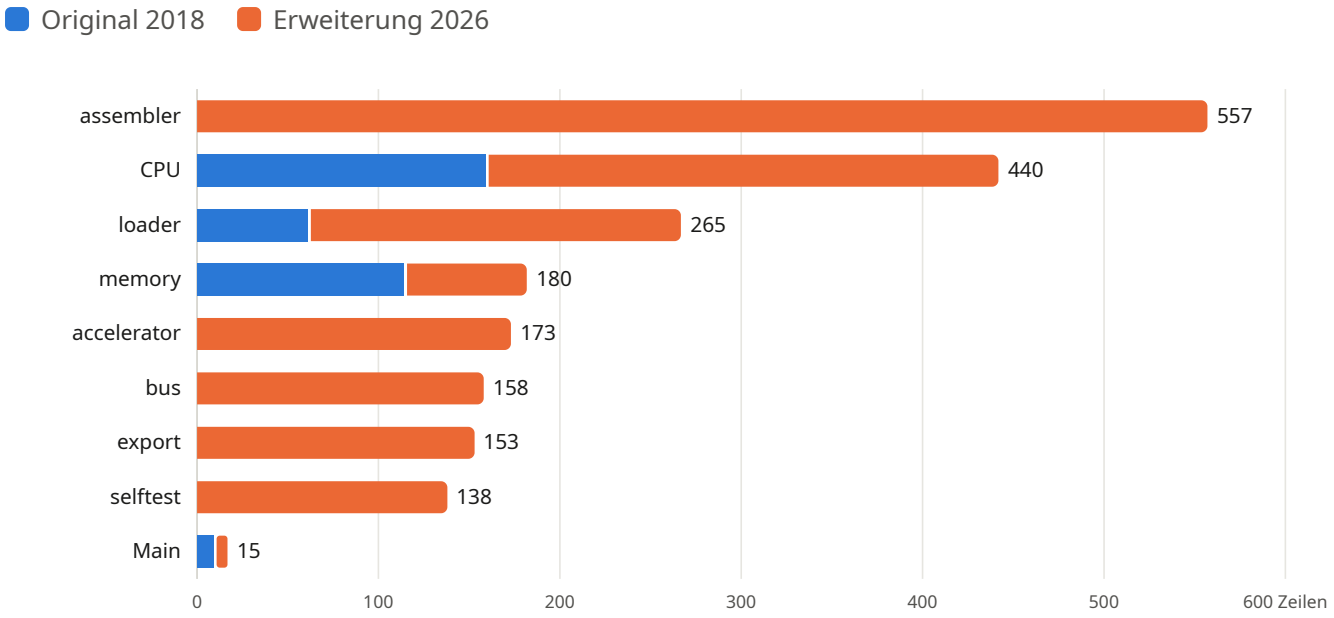


Abbildung 1: Java-Codezeilen je Paket im aktuellen Stand, aufgeteilt nach Herkunft. Die Pakete `CPU`, `loader`, `memory` und `Main` enthalten beides.

Paket	Original 2018	Erweiterung 2026	Summe
assembler	0	557	557
CPU	159	281	440
loader	61	204	265
memory	114	66	180
accelerator	0	173	173
bus	0	158	158
export	0	153	153
selftest	0	138	138
Main (Standardpaket)	9	6	15
Gesamt	**343**	**1 736**	**2 079**

Änderungen an Dateien aus 2018

Vier Originaldateien wurden angepasst. Die folgende Tabelle beschreibt, was dabei vom Original stammt und was ergänzt wurde.

Datei	erhalten	ergänzt	Original 2018	Erweiterung 2026
`loader/Loader.java` **gemischt**	61	204	Menüführung (Testmodus, CPU-Auswahl, Dateiname), ANSI-Farbausgabe, Laden aus dem Klassenpfad unter `Programm/`, Entfernen von Kommentaren und Leerzeilen, Testmodus-Ausgabe. Der Menüpunkt 2 „CPU mit Coprozessor“ war bereits als Platzhalter vorgesehen.	Menüpunkt 2 startet den cristian1604-MIPS mit Bus und Beschleuniger; Abfrage von Latenz und Nachkommabits; Hex-Ausgabe und Export; Kommandozeilenmodus `runBatch`; zusätzliche Suche im Dateisystem; Auslagerung von Einlesen und Kommentarentfernung in Methoden.
`CPU/AbstractCPU.java` **gemischt**	12	17	Klassengerüst mit dem Kommentar „Gemeinsame Methoden (optional)“.	Hilfsmethoden `intToBits` / `bitsToInt`. Ihre Logik entspricht den privaten Methoden aus `simpleCPU` (2018) und wurde zur Wiederverwendung hierher übertragen.
`memory/Register.java` **gemischt**	110	2	Komplettes Registerfeld mit Bit-Arrays, Schutz von `$0`, farbige Ausgabe.	Registerausgabe zeigt zusätzlich den Hex-Wert.
`Main.java` **gemischt**	9	6	Einstiegspunkt, der den Loader startet.	Mit Programmargumenten wird der Kommandozeilenmodus gestartet.

Kleine redaktionelle Änderungen des Autors außerhalb der Erweiterung: Entfernen der IntelliJ-Vorlagenkommentare in `Main.java` und zweier Leerzeilen in `AbstractCPU.java`.

Was die Erweiterung vom Original wiederverwendet

- **Schnittstelle** `CPU` und **Basisklasse** `AbstractCPU` – die neue CPU fügt sich in die vorhandene Architektur ein, wie es 2018 vorgesehen war.
- **Registerfeld** `memory.Register` – die neue CPU speichert ihre Register in derselben Bit-Array-Struktur und nutzt dieselbe Ausgabe.
- **Menü des Loaders** – der 2018 angelegte Platzhalter „CPU mit Coprozessor“ wurde mit der neuen CPU gefüllt.
- **Programmverzeichnis** `src/Programm/` und das Laden über den Klassenpfad.

2 Systemüberblick

Abbildung 2 zeigt alle Pakete und Klassen mit ihrer Herkunft. Die Originalklassen bilden weiterhin einen eigenständigen, lauffähigen Lern-Emulator (Menüpunkt 1). Die Erweiterung ergänzt einen zweiten, unabhängigen Ausführungspfad (Menüpunkt 2), der auf Maschinencode-Ebene arbeitet.

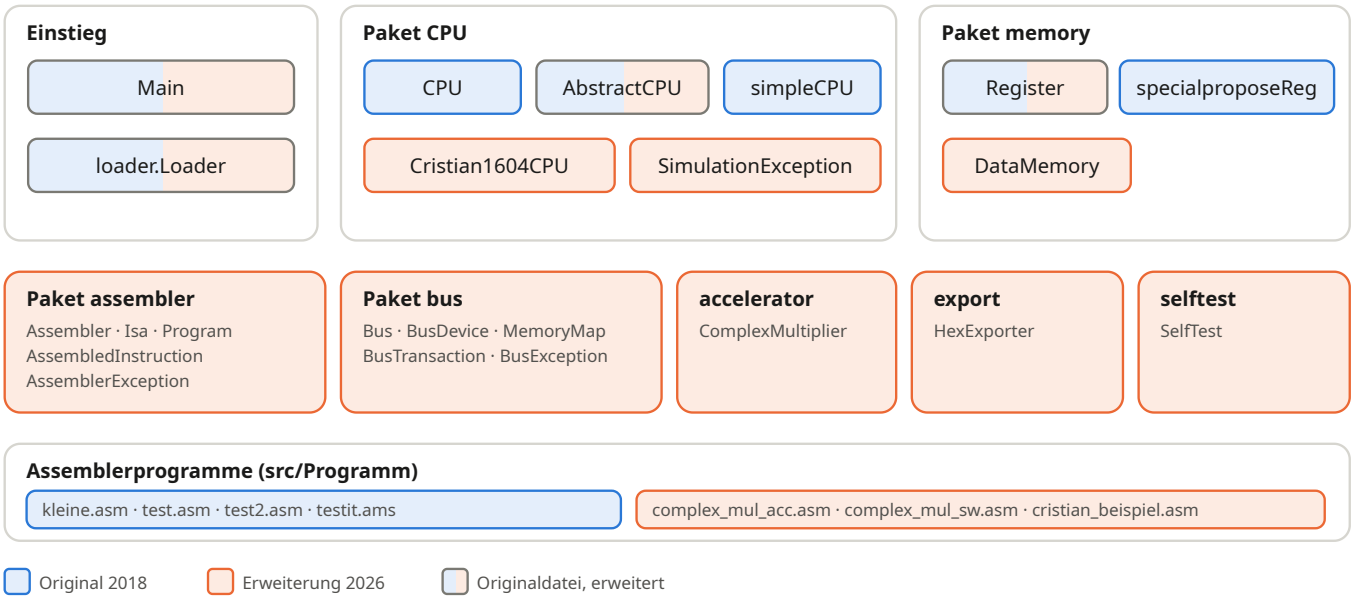


Abbildung 2: Paketübersicht mit Herkunftskennzeichnung.

3 Die Originalimplementierung von 2018 Original 2018

> Dieses Kapitel beschreibt ausschließlich den händisch geschriebenen Stand von 2018. Der Code ist unverändert weiterhin enthalten und über Menüpunkt 1 („Standard CPU“) nutzbar.

Zielsetzung

Der Emulator sollte möglichst schnell eine funktionsfähige MIPS-Emulation bereitstellen, die sich auch Anfängerinnen und Anfängern mit wenig Java-Erfahrung erschließt. Vorbild war MARS, wobei bewusst ein einfacheres Modell gewählt wurde: Der Emulator interpretiert die Assemblerzeilen direkt als Text, statt sie in Maschinencode zu übersetzen.

Ablauf

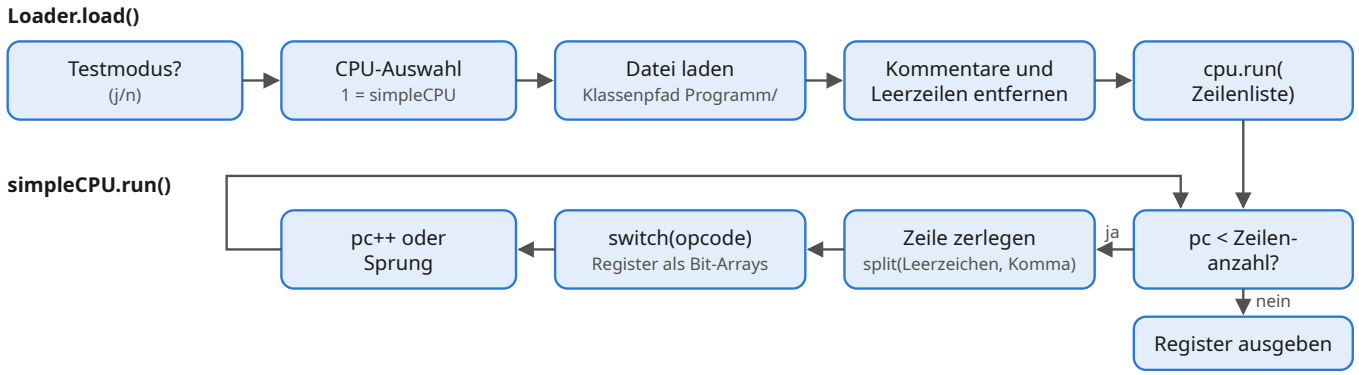


Abbildung 3: Ablauf der Originalimplementierung. Jede Zeile wird bei jeder Ausführung erneut als Text zerlegt und interpretiert.

Komponenten

Klasse	Aufgabe
`Main`	Einstiegspunkt, startet den Loader.
`loader.Loader`	Konsolenmenü (Testmodus, CPU-Wahl, Dateiname), liest die Datei aus dem Klassenpfad unter `Programm/`, entfernt Kommentare (`#`) und Leerzeilen, übergibt die Zeilen an die CPU und gibt im Testmodus die gelesenen Instruktionen in Orange aus.
`CPU.CPU`	Schnittstelle mit der Methode `run(List<String>)`.
`CPU.AbstractCPU`	Vorgesehene Basisklasse für gemeinsame Methoden (2018 noch leer).

Klasse	Aufgabe
`CPU.simpleCPU`	Interpretiert die Textzeilen: Programmzähler als Zeilenindex, Befehle per `switch`, Speicher als `HashMap<Integer,Integer>`.
`memory.Register`	32 Register zu je 32 Bit, gespeichert als `int[32][32]` (MSB an Index 0). `$0` ist schreibgeschützt; Ausgabe binär (grün) und dezimal.
`memory.specialproposeReg`	Platzhalter für Spezialregister (leer).

Befehlsumfang und Semantik

Befehl	Semantik in `simpleCPU`
`add, sub, and, or, slt`	Registeroperationen `rd = rs ∘ rt`
`addi rt, rs, imm`	`rt = rs + imm` (Dezimalzahl)
`lw / sw rt, off(rs)`	Zugriff auf die HashMap mit Schlüssel `rs + off`
`beq / bne rs, rt, n`	bei Erfüllung `pc = pc + n` (Sprung um *n* Zeilen ab der aktuellen Zeile)
`j n`	`pc = n` (Zeilenindex)

Register werden ausschließlich numerisch angegeben (`$5`). Labels gibt es nicht; Sprungziele werden als Zeilenzahlen nach dem Entfernen von Kommentaren und Leerzeilen angegeben.

Stärken und Grenzen

Stärken

- Sehr kompakt (139 Zeilen CPU) und direkt nachvollziehbar
- Register als Bit-Arrays machen die Binärdarstellung sichtbar – didaktisch wertvoll
- Saubere Trennung von Loader, CPU-Schnittstelle und Registerfeld; die Erweiterung konnte an diese Struktur andocken
- Testmodus und farbige Ausgabe erleichtern das Nachvollziehen

Grenzen (Anlass für die Erweiterung)

- Kein Maschinencode – ein Hex-Export für Hardware ist nicht möglich
- Sprungsemantik weicht von MIPS ab (Zeilen statt Wörter relativ zu PC+4)
- Keine Labels, keine symbolischen Registernamen
- Fehler werden gemeldet, die Ausführung läuft aber weiter
- Keine Peripherie bzw. kein Bus, kein Taktmodell
- Keine Begrenzung der Zyklen: `kleine.asm` gerät durch `j 2` (zurück zu `slt`) in eine Endlosschleife

4 Zielplattform cristian1604/mips

Erweiterung 2026

Als Ziel-CPU für die Bachelorarbeit dient der Single-Cycle-MIPS von Cristian Bottazzi (github.com/cristian1604/mips, VHDL, GPL-3.0). Er folgt dem Entwurf aus Harris & Harris, *Digital Design and Computer Architecture*. Die Erweiterung des Emulators bildet diesen Prozessor nach. Grundlage dafür war die Analyse der VHDL-Quellen.

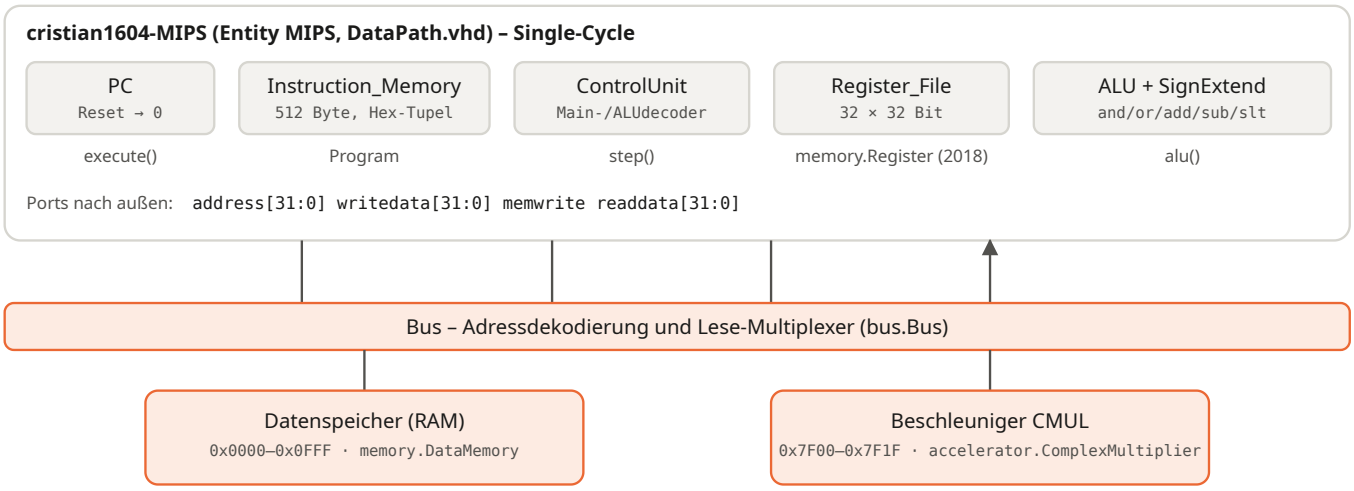


Abbildung 4: Zielsystem. Graue Blöcke sind Teile des cristian1604-Prozessors (VHDL), darunter jeweils die Java-Klasse bzw. Methode, die den Block im Emulator nachbildet. Datenspeicher und Beschleuniger liegen außerhalb des Prozessors am Bus.

Befehlssatz der Hardware

`MainDecoder.vhd` und `ALUdecoder.vhd` dekodieren nur die folgenden Befehle. Alles andere erzeugt in VHDL undefinierte Steuersignale (`"---"`).

Befehl	Format	Opcode	Funct	Wirkung im Datenpfad
`add rd, rs, rt`	R	`0x00`	`0x20`	rd = rs + rt (kein Überlauf-Trap)
`sub rd, rs, rt`	R	`0x00`	`0x22`	rd = rs − rt
`and rd, rs, rt`	R	`0x00`	`0x24`	rd = rs & rt
`or rd, rs, rt`	R	`0x00`	`0x25`	rd = rs | rt
`slt rd, rs, rt`	R	`0x00`	`0x2A`	rd = Vorzeichenbit von (rs − rt)
`addi rt, rs, imm`	I	`0x08`	–	rt = rs + SignExt(imm)
`lw rt, imm(rs)`	I	`0x23`	–	rt = readdata, address = rs + SignExt(imm)
`sw rt, imm(rs)`	I	`0x2B`	–	writedata = rt, memwrite = 1
`beq rs, rt, imm`	I	`0x04`	–	falls rs − rt = 0: PC = PC + 4 + (SignExt(imm) << 2)

Befehl	Format	Opcode	Funct	Wirkung im Datenpfad
`j addr`	J	`0x02`	–	PC = (PC + 4)[31:28] & addr & "00"

Besonderheiten, die der Emulator nachbildet

- **Textsegment ab Adresse 0.** `PC.vhd` setzt den PC beim Reset auf 0. MARS legt Programme ab `0x00400000` ab und kodiert `j`-Befehle deshalb für diese CPU falsch (das README von cristian1604 empfiehlt, `x"08",x"10"` von Hand durch `x"08",x"00"` zu ersetzen). Der Emulator erzeugt direkt richtigen Code.
- **`slt` ohne Überlaufkorrektur.** `ALU.vhd` liefert das Vorzeichenbit von `a - b`. Läuft die Subtraktion über, ist das Ergebnis falsch. Der Emulator rechnet wie die Hardware und gibt in diesem Fall eine Warnung aus.
- **Kein MemRead-Signal.** Am Port `address` liegt bei jedem Befehl das ALU-Ergebnis an. Peripherie darf deshalb beim Lesen keinen Zustand ändern (siehe Kapitel 9).
- **Kein `lui` / `ori`.** Konstanten sind auf 16 Bit mit Vorzeichen begrenzt. Das bestimmt die Wahl der Peripherie-Adressen (Kapitel 8).
- **Kein `nop` als Nullwort.** `0x00000000` (sll) kennt der ALUdecoder nicht; der Pseudobefehl `nop` wird deshalb als `add $zero, $zero, $zero` (`0x00000020`) kodiert.
- **Kein Halt-Befehl.** Programme enden mit einem Sprung auf sich selbst.

5 Verarbeitungskette der Erweiterung

Erweiterung 2026

Der zentrale Unterschied zur Originalimplementierung: Das Programm wird zuerst vollständig in 32-Bit- Maschinenwörter übersetzt, und die CPU führt *diese Wörter* aus. Dadurch ist garantiert, dass der exportierte Hex-Code exakt dem simulierten Programm entspricht. Der Export erfolgt nur, wenn die Simulation fehlerfrei abgeschlossen wurde.

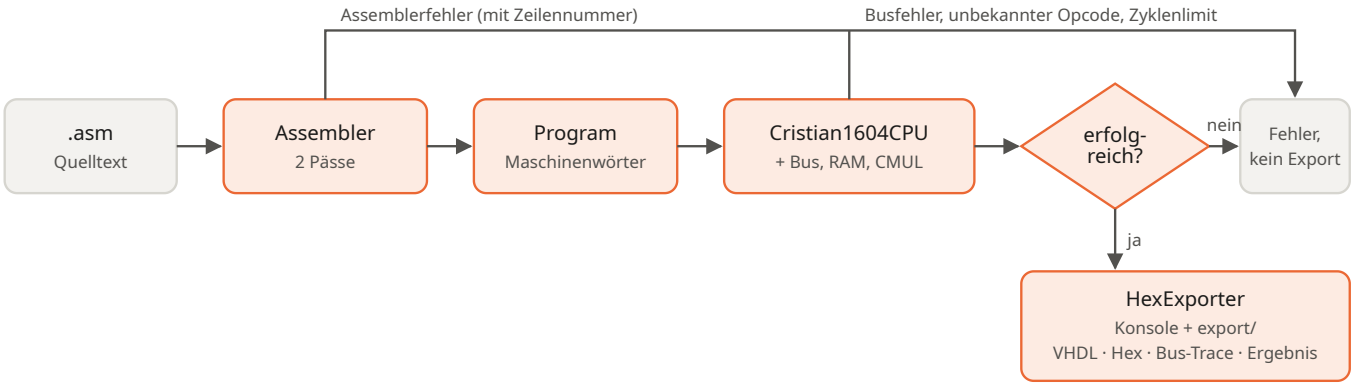


Abbildung 5: Verarbeitungskette. Assemblerfehler und Laufzeitfehler führen dazu, dass kein Hex-Code ausgegeben wird.

6 Assembler Erweiterung 2026

Zwei-Pass-Verfahren

`assembler.Assembler` arbeitet in zwei Durchläufen:

1. **Pass 1** entfernt Kommentare, sammelt Labels und `.eqv`-Konstanten und vergibt jeder Anweisung ihre Adresse. Pseudobefehle, die zu mehreren Maschinenbefehlen werden (`bne`), belegen entsprechend mehr Platz.
2. **Pass 2** kodiert jede Anweisung. Weil alle Labels bekannt sind, sind auch Vorwärtssprünge möglich.

Fehler werden nicht beim ersten Auftreten abgebrochen, sondern mit Zeilennummer gesammelt und gemeinsam ausgegeben. Die Klasse `Isa` enthält die Opcodes, Funct-Codes und Registernamen sowie einen Disassembler, der Maschinenwörter in wieder einlesbare Assembler-Syntax zurückübersetzt.

Befehlsformate

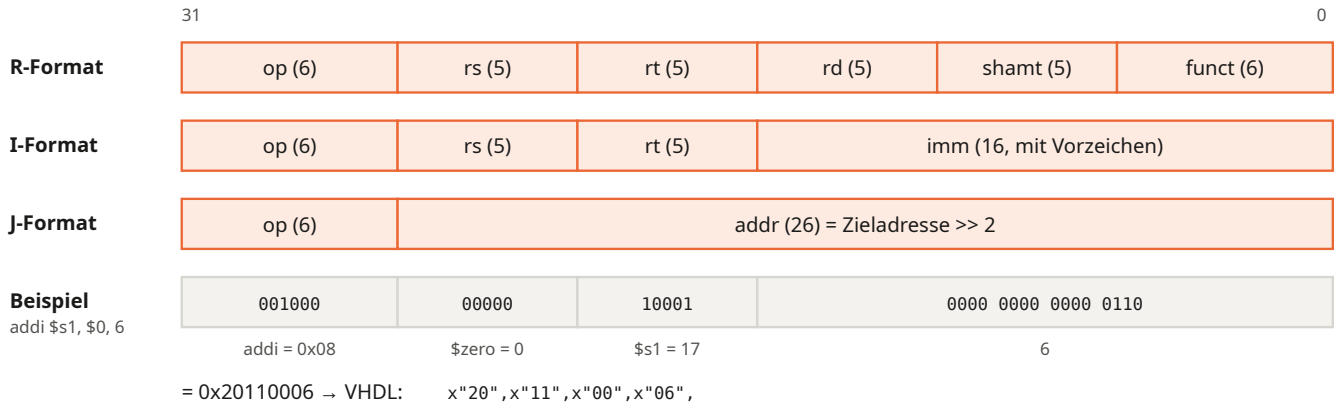


Abbildung 6: Befehlsformate (Bitbreiten maßstäblich) und die Kodierung des ersten Befehls aus dem Beispielprogramm von cristian1604. Der Emulator erzeugt exakt den dort angegebenen Hex-Code.

Sprache

Element	Syntax
Kommentar	`# …`
Label	`name:`, auch vor einem Befehl in derselben Zeile
Register	`$0` … `$31` oder `$zero, $at, $v0–$v1, $a0–$a3, $t0–$t9, $s0–$s7, $k0–$k1, $gp, $sp, $fp, $ra`
Zahlen	dezimal, `0x…`, `0b…`; Ausdrücke mit `+` und `-`

Element	Syntax
Konstanten	`.eqv NAME WERT` (MARS-kompatibel)
Ignoriert	`.text`, `.globl`
Abgelehnt	`.data` (Datenspeicher ist extern), alle Befehle außerhalb des Hardware-Befehlssatzes
`beq`-Ziel	Label oder Zahl = Offset in Wörtern relativ zu PC + 4
`j`-Ziel	Label oder Byte-Adresse
Immediate	−32 768 … 32 767

Pseudobefehle

Pseudobefehl	wird übersetzt in	Begründung
`nop`	`add $zero, $zero, $zero`	Nullwort ist nicht dekodierbar
`move rd, rs`	`add rd, rs, $zero`	Komfort
`li rt, imm16`	`addi rt, $zero, imm16`	nur 16 Bit, kein `lui`
`b label`	`beq $zero, $zero, label`	Komfort
`bne rs, rt, label`	`beq rs, rt, 1` + `j label`	Hardware hat kein `bne`
`halt`	`j` auf die eigene Adresse	Hardware hat keinen Halt-Befehl

7 CPU-Emulation Erweiterung 2026

`CPU.Cristian1604CPU` erbt von `AbstractCPU` und implementiert damit die Schnittstelle `CPU` aus 2018. Pro Takt wird ein Befehl ausgeführt:

1. Prüfen, ob der PC innerhalb des Programms liegt und das Zyklenlimit nicht erreicht ist
2. Maschinenwort holen und in die Felder `opcode, rs, rt, rd, funct, imm` zerlegen
3. Ausführen wie im Datenpfad: ALU-Operation, Speicherzugriff über den Bus oder Sprung
4. Taktflanke an alle Busgeräte (`bus.tick()`), Taktzähler erhöhen
5. Ist der nächste PC gleich dem aktuellen (Sprung auf sich selbst), endet die Simulation erfolgreich

```
case Isa.OP_BEQ:
    // pcsrc = branch and zero; zero kommt aus der ALU-Subtraktion
    if (readReg(rs) - readReg(rt) == 0) {
        nextPc = pcPlus4 + (signImm << 2);
    }
    break;
case Isa.OP_J:
    nextPc = (pcPlus4 & 0xF0000000) | ((instr & 0x03FFFFFF) << 2);
    break;
```

Ausgaben

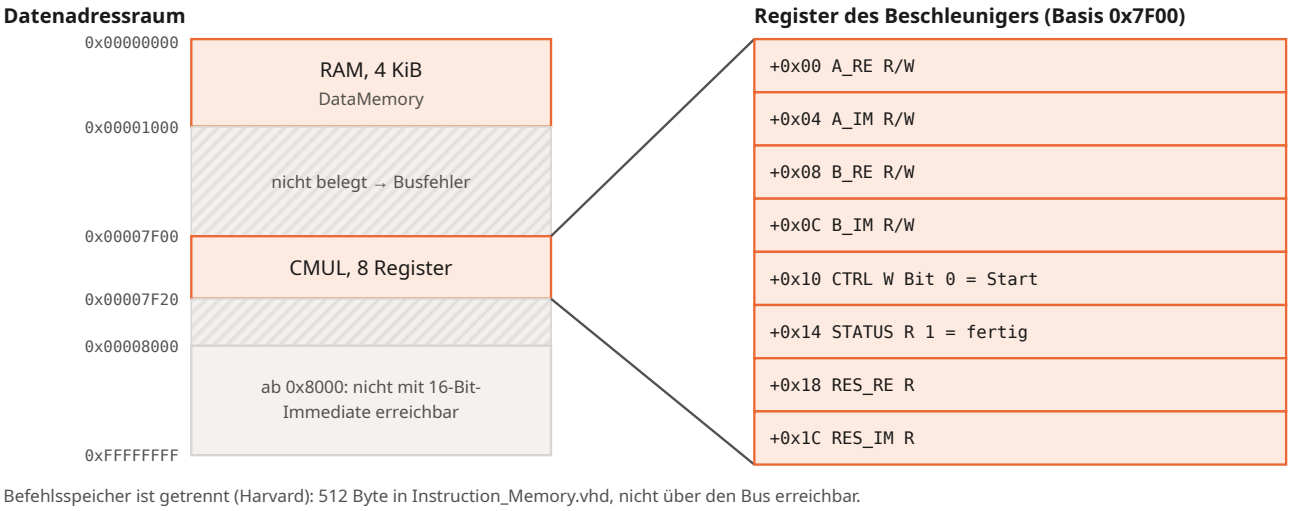
- Registerinhalte über `Register.printAllRegisters()` (2018, ergänzt um Hex)
- Statistik: Takte, Anzahl je Befehl, Anzahl Buszugriffe, Art des Programmendes
- Zustand aller Busgeräte (RAM-Inhalt, Register des Beschleunigers)
- Warnungen, z. B. `slt`-Überlauf oder fehlendes `halt`
- Im Testmodus bzw. mit `--trace`: jeder Befehl mit Takt, PC, Maschinenwort, Disassemblat und Buszugriffen

Fehlerbehandlung

Anders als in `simpleCPU` bricht jeder Fehler die Simulation ab: nicht ausgerichtete Zugriffe, Adressen ohne Gerät, Schreiben auf nur lesbare Register, nicht dekodierbare Befehle und das Erreichen des Zyklenlimits (Standard 1 000 000). Die Meldung enthält PC, Quellzeile und Ursache.

8 Bus und Speicherbelegung Erweiterung 2026

`bus.Bus` verbindet die CPU mit Geräten, die die Schnittstelle `BusDevice` implementieren. Der Bus dekodiert die Adresse, leitet den Zugriff mit dem Offset relativ zur Basisadresse an das Gerät weiter, protokolliert jede Transaktion (`BusTransaction`) und verteilt die Taktflanke. Überlappende Adressbereiche werden beim Anschließen abgewiesen.



Befehlsspeicher ist getrennt (Harvard): 512 Byte in Instruction_Memory.vhd, nicht über den Bus erreichbar.

Abbildung 7: Speicherbelegung (nicht maßstäblich). Die Basisadressen stehen in `bus/MemoryMap.java` .

Wahl der Adressen

Da der Prozessor weder `lui` noch `ori` kennt, lassen sich Adressen nur über ein vorzeichenbehaftetes 16-Bit-Immediate bilden, z. B. `addi $s0, $zero, 0x7F00` oder direkt `lw $t0, 0x7F14($zero)` . Der Beschleuniger liegt deshalb unterhalb von `0x8000` .

Vorschlag für die Adressdekodierung in VHDL

Signal	Bedingung
`ram_sel`	`address(31 downto 12) = x"00000"`
`acc_sel`	`address(31 downto 5) = 0x3F8` (entspricht `0x7F00–0x7F1F`)
Registerauswahl	`address(4 downto 2)`
`readdata`	Multiplexer: RAM-Daten bei `ram_sel` , Beschleunigerregister bei `acc_sel`
Schreibfreigabe	`memwrite and ram_sel` bzw. `memwrite and acc_sel`

9 Beschleuniger für komplexe Multiplikation Erweiterung 2026

`accelerator.ComplexMultiplier` ist das Verhaltensmodell („Golden Model“) des in der Bachelorarbeit zu entwickelnden Hardware-Beschleunigers. Er berechnet

```
(a_re + j·a_im) · (b_re + j·b_im)
    = (a_re·b_re − a_im·b_im) + j·(a_re·b_im + a_im·b_re)
```

Zustandsautomat

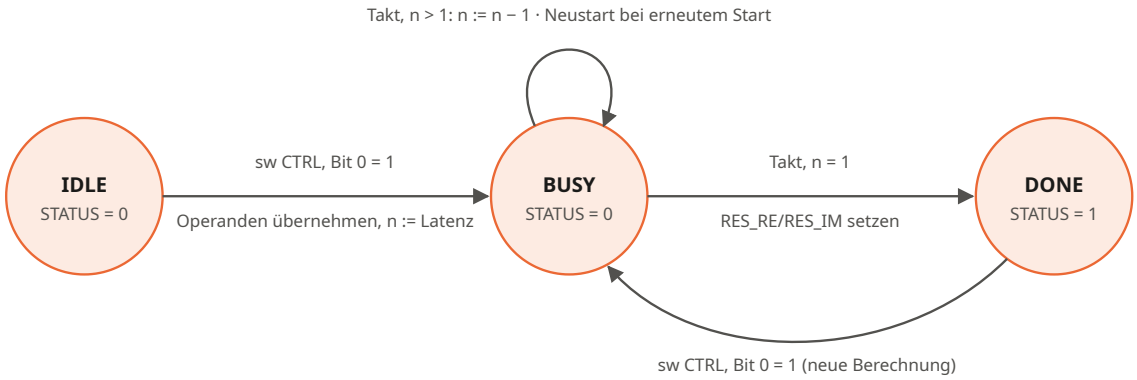


Abbildung 8: Zustände des Beschleunigers. Operandenregister können jederzeit beschrieben werden; übernommen werden sie erst beim Start.

Zeitverhalten am Bus

Abbildung 9 zeigt einen Ausschnitt aus der Simulation von `complex_mul_acc.asm` mit Latenz 4. Die Werte stammen aus dem exportierten Bus-Trace.

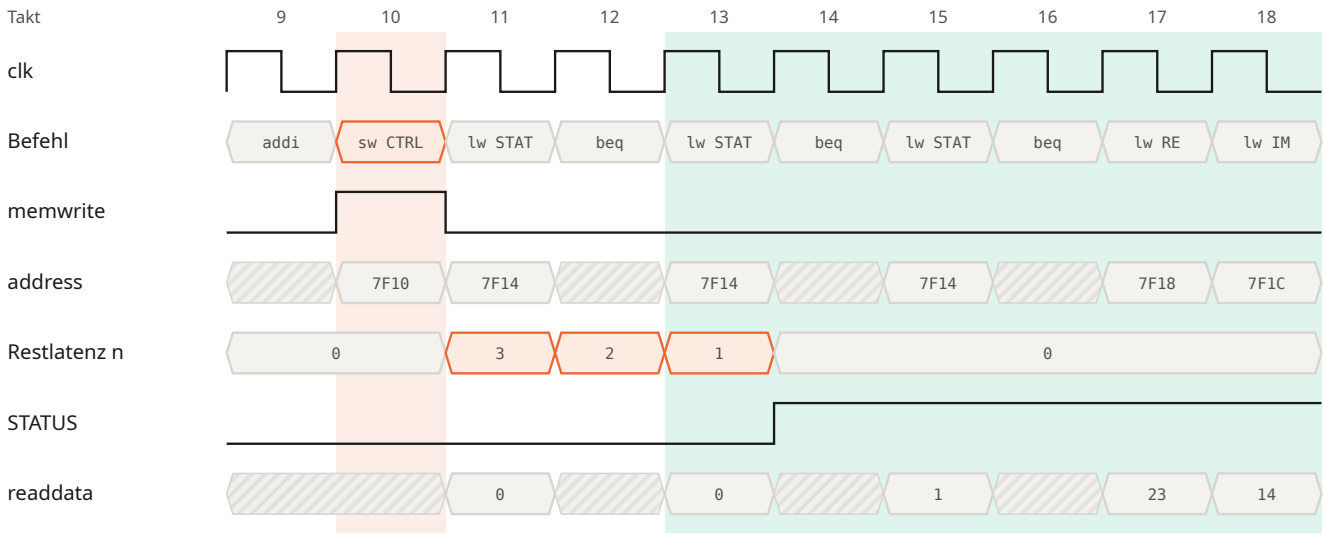


Abbildung 9: Zeitdiagramm (Takte 9–18, Latenz 4). In Takt 10 startet `sw` auf CTRL die Berechnung (orange hinterlegt). Nach vier Taktflanken ist ab Takt 14 `STATUS = 1` (grün hinterlegt). Schraffiert: Der Bus führt das

ALU-Ergebnis eines Befehls ohne Speicherzugriff – ein Wert, den die Peripherie ignorieren muss.

Entwurfsentscheidungen

Entscheidung	Begründung
STATUS ist ein einzelnes Bit (0/1)	Die CPU hat nur `beq` und kein `andi`. Die Warteschleife `lw` + `beq $t5, $zero, poll` funktioniert nur, wenn STATUS im Wartezustand genau 0 ist.
Lesen ohne Seiteneffekte	Ohne MemRead-Signal liegt bei jedem Befehl eine Adresse am Bus an. Ein „Löschen beim Lesen“ würde durch beliebige Befehle ausgelöst.
Start über eigenes Register CTRL	Eindeutiger Startzeitpunkt; Operanden können in beliebiger Reihenfolge geschrieben werden.
Latenz einstellbar (≥ 1)	Erlaubt, verschiedene Hardware-Varianten (kombinatorisch, gepipelined, sequentiell) vorab zu bewerten.
Festkomma mit `fracBits` Nachkommabits	0 = Ganzzahl, 15 = Q15 usw. Produkte in voller Breite, arithmetischer Rechtsshift (Abrundung), Abschneiden auf 32 Bit ohne Sättigung.
Schreiben auf nur lesbare Register = Fehler	Deckt Programmierfehler im Emulator auf; in Hardware würde der Zugriff ignoriert.

Die statische Methode `ComplexMultiplier.multiply(aRe, aIm, bRe, bIm, fracBits)` liefert die Erwartungswerte auch ohne Simulation, z. B. für Testvektoren einer VHDL-Testbench.

10 Hex-Export Erweiterung 2026

`export.HexExporter` gibt nach erfolgreicher Simulation den Code auf der Konsole aus und schreibt fünf Dateien:

Datei	Inhalt und Verwendung
`<name>_Instruction_Memory.vhd`	Vollständiger Ersatz für `Instruction_Memory.vhd` im cristian1604-Projekt, inklusive Quelltext als Kommentar
`<name>_vhdl.txt`	Nur die Byte-Tupel zum Einfügen in eine bestehende Datei
`<name>.hex`	Ein 32-Bit-Wort pro Zeile (wie MARS „Hexadecimal Text“, z. B. für `$readmemh`)
`<name>_bus_trace.txt`	Erwartete Buszugriffe mit Takt, PC, R/W, Adresse, Daten – zum Vergleich mit der Testbench
`<name>_ergebnis.txt`	Endzustand: Taktzahl, Register ≠0, RAM-Inhalt, Beschleunigerzustand

Ausschnitt `complex_mul_acc_Instruction_Memory.vhd`

```
x"20",x"10",x"7f",x"00",  -- 0x0000: addi $s0, $zero, 32512
x"20",x"08",x"00",x"03",  -- 0x0004: addi $t0, $zero, 3
…
x"ae",x"0c",x"00",x"10",  -- 0x0028: sw $t4, 16($s0)
x"8e",x"0d",x"00",x"14",  -- 0x002c: lw $t5, 20($s0)
x"11",x"a0",x"ff",x"fe",  -- 0x0030: beq $t5, $zero, -2
…
x"08",x"00",x"00",x"11",  -- 0x0044: j 0x00000044

others=> x"00");
```

Ausschnitt `complex_mul_acc_bus_trace.txt`

```
#     Takt  PC          R/W  Adresse     Daten            (dez)  Gerät
        10  0x00000028  W  0x00007f10  0x00000001           1  CMUL
        11  0x0000002c  R  0x00007f14  0x00000000           0  CMUL
        15  0x0000002c  R  0x00007f14  0x00000001           1  CMUL
        17  0x00000034  R  0x00007f18  0x00000017          23  CMUL
        18  0x00000038  R  0x00007f1c  0x0000000e          14  CMUL
```

Programme über 512 Byte (128 Befehle) erzeugen eine Warnung; das Array in der erzeugten VHDL-Datei wird dann vergrößert.

11 Klassenstruktur

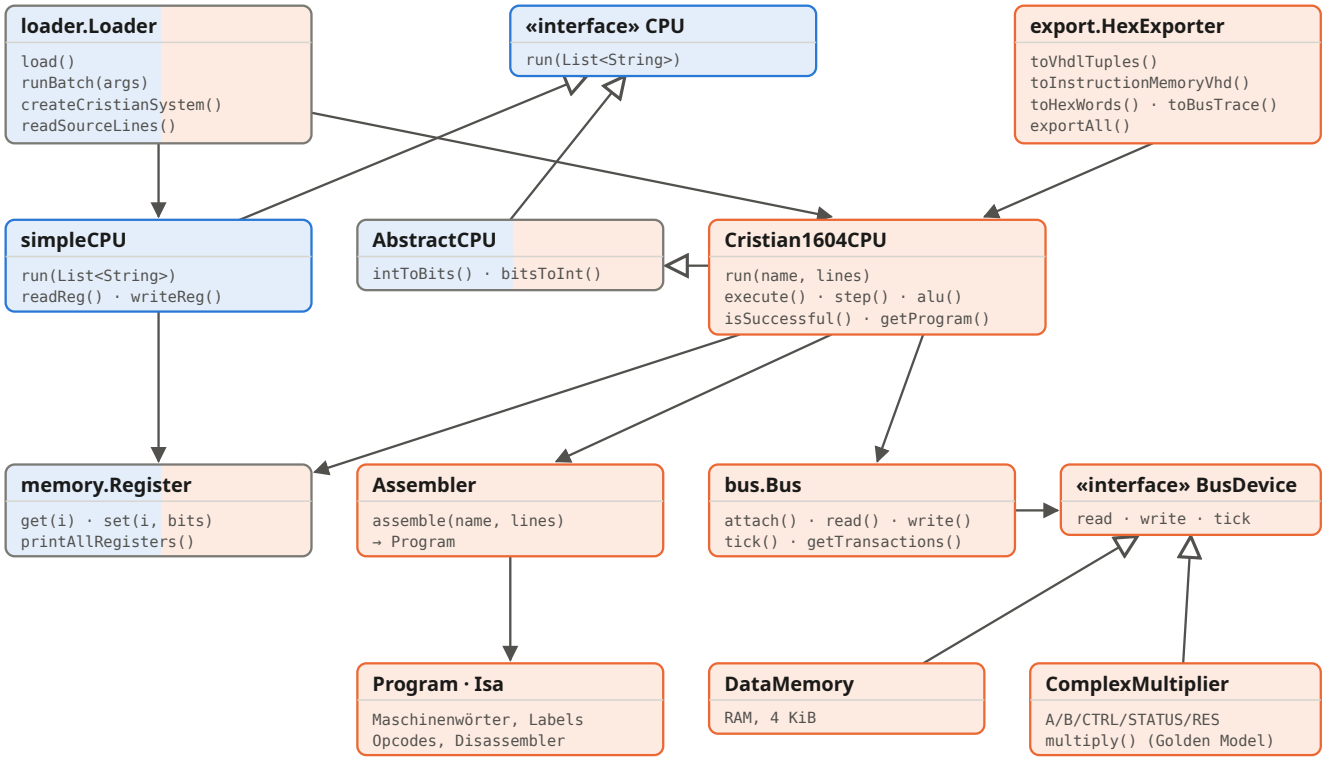


Abbildung 10: Vereinfachtes Klassendiagramm. Hohle Dreiecke: Vererbung bzw. Implementierung; offene Pfeile: Verwendung. Farbe wie in Abbildung 2 (blau = 2018, orange = 2026, geteilt = erweitert).

12 Bedienung

Übersetzen und Starten

```
javac -encoding UTF-8 -d out $(find src -name '*.java')

java -cp out Main                                  # Menü (wie 2018)
java -cp out Main complex_mul_acc.asm              # ohne Rückfragen, Export nach expor
java -cp out Main complex_mul_acc.asm --latency 8 --trace
java -cp out selftest.SelfTest                     # Selbsttest
```

In IntelliJ wird `Main` wie bisher gestartet.

Interaktives Menü gemischt

```
Testmodus aktivieren? (j/n): n
Willkommen zum MIPS-Emulator!
Bitte wählen Sie eine CPU-Variante:
1 - Standard CPU (vereinfachte Lern-CPU)
2 - cristian1604-MIPS mit Beschleuniger für komplexe Multiplikation am Bus
Ihre Wahl: 2
Name der zu ladenden .asm-Datei (z. B. 'xy.asm'): complex_mul_acc.asm
Latenz des Beschleunigers in Takten [4]:
Nachkommabits der Festkommadarstellung (0 = Ganzzahl) [0]:
…
Hex-Dateien nach 'export/' schreiben? (j/n): j
```

Testmodus, CPU-Auswahl und Dateiname stammen aus der Originalimplementierung. Die Abfragen von Latenz, Nachkommabits und Export gehören zur Erweiterung. Bei Menüpunkt 2 schaltet der Testmodus zusätzlich den Trace jedes Befehls ein.

Kommandozeilenoptionen Erweiterung 2026

Option	Bedeutung	Standard
`--cpu simple|cristian`	alte Lern-CPU oder cristian1604-MIPS	`cristian`
`--latency N`	Takte bis der Beschleuniger fertig ist	4
`--frac N`	Nachkommabits (0 = Ganzzahl)	0
`--max-cycles N`	Zyklenlimit	1 000 000
`--trace`	jeden Befehl und Buszugriff ausgeben	aus

Option	Bedeutung	Standard
`--export DIR`	Zielverzeichnis	`export`
`--no-export`	nur Konsolenausgabe	–

Exit-Code 0 bei erfolgreicher Simulation, sonst 1. Die `.asm`-Datei wird zuerst im Klassenpfad unter `Programm/` gesucht (wie 2018), dann im Dateisystem.

13 Beispielprogramme und Ergebnisse Erweiterung 2026

Programm	Inhalt	Befehle	Takte
`complex_mul_acc.asm`	(3 + 4j)·(5 − 2j) mit dem Beschleuniger	18	22*
`complex_mul_sw.asm`	dasselbe in Software, Multiplikation durch wiederholte Addition	53	93
`cristian_beispiel.asm`	6 · 4 aus dem cristian1604-README, ergänzt um `halt`	9	30

* bei Latenz 4

Beide Varianten der komplexen Multiplikation liefern 23 + 14j in RAM[0] und RAM[4]. Abbildung 11 zeigt, wie sich die Latenz des Beschleunigers auf die Gesamttaktzahl auswirkt.

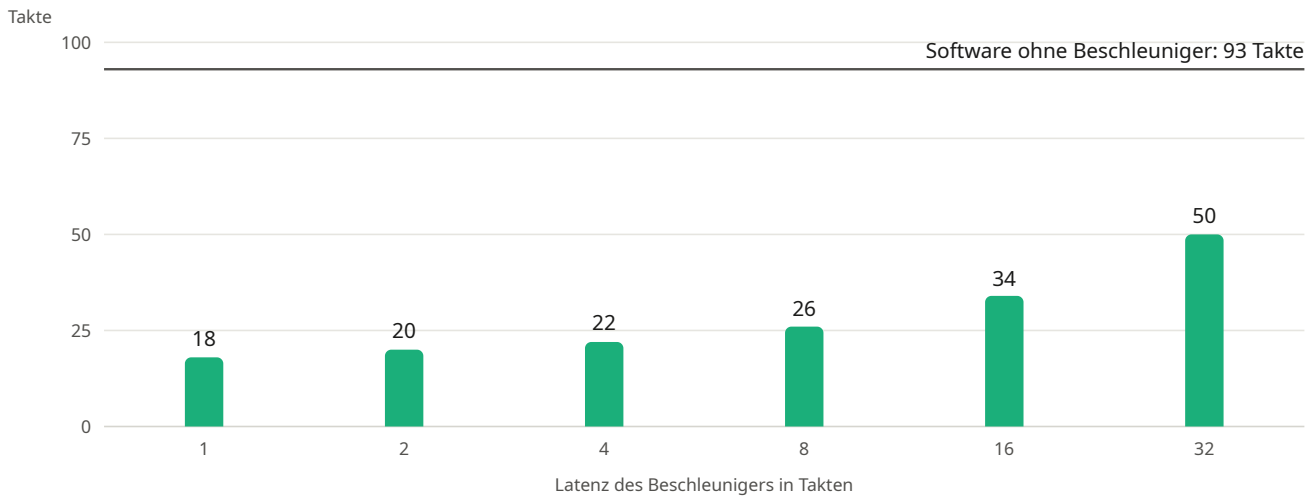


Abbildung 11: Gesamttakte von `complex_mul_acc.asm` bei verschiedenen Latenzen, verglichen mit der Software-Variante (Linie). Die Taktzahl wächst um etwa einen Takt pro Latenztakt, weil die Warteschleife aus zwei Befehlen besteht.

Latenz	Takte mit Beschleuniger	Takte Software	Beschleunigung
1	18	93	5,2×
2	20	93	4,7×
4	22	93	4,2×
8	26	93	3,6×
16	34	93	2,7×
32	50	93	1,9×

Die Software-Taktzahl hängt von den Operanden ab (wiederholte Addition, etwa drei Takte pro Einheit des Multiplikators). Bei größeren Operanden fällt der Vorteil des Beschleunigers deutlich höher aus; die Werte gelten für das Beispiel (3 + 4j)·(5 − 2j).

14 Test und Verifikation Erweiterung 2026

`selftest.SelfTest` kommt ohne externes Test-Framework aus und prüft:

Test	Prüft	Ergebnis
Kodierung README-Beispiel	Alle 8 Befehle des cristian1604-Beispiels Byte für Byte gegen den dort veröffentlichten Hex-Code	bestanden
Roundtrip	Disassemblieren und erneutes Assemblieren ergibt dasselbe Maschinenwort (18 Befehle)	bestanden
Nicht unterstützte Befehle	`mult` wird mit Fehlermeldung abgelehnt	bestanden
Beschleuniger-Programm	23 + 14j bei Latenz 1, 4 und 16	bestanden
Software-Referenz	23 + 14j ohne Beschleuniger	bestanden
Festkomma	$(0{,}5 + 0{,}5j)^2 = 0{,}5j$ im Q15-Format	bestanden

Zusätzlich manuell geprüft: Fehlerpfade (Assemblerfehler, Busfehler auf unbelegter Adresse, Programm ohne `halt`), Exit-Codes im Kommandozeilenmodus, das interaktive Menü mit beiden CPUs und die Standard-CPU mit `test2.asm` (unverändertes Verhalten).

Empfohlener Abgleich mit der Hardware

1. Programm im Emulator simulieren und exportieren
2. `<name>_Instruction_Memory.vhd` im cristian1604-Projekt einsetzen
3. In der VHDL-Testbench alle Zugriffe mit `memwrite = '1'` sowie die Lesezugriffe auf den Beschleuniger mit Taktnummer protokollieren
4. Protokoll mit `<name>_bus_trace.txt` vergleichen, Endzustand mit `<name>_ergebnis.txt`

15 Grenzen und Ausblick

Bekannte Grenzen Erweiterung 2026

- Das Taktmodell bildet einen idealen Single-Cycle-Prozessor ab; Laufzeiten und Setup-/Hold-Bedingungen der realen Schaltung werden nicht modelliert.
- Registerwerte und Speicher sind nach Reset 0. In VHDL ist `Register_File` nicht initialisiert (`'U'`) – Programme sollten Register vor dem Lesen beschreiben.
- Kein `.data`-Segment: Daten müssen per `addi` / `sw` erzeugt werden.
- Speicherzugriffe nur wortweise.
- Registerbelegung, Adressen und Zahlenformat des Beschleunigers sind ein begründeter Vorschlag und müssen mit dem endgültigen Hardware-Entwurf abgeglichen werden.

Mögliche Erweiterungen

- Gepackte Operanden (Real- und Imaginärteil je 16 Bit in einem Wort), um Buszugriffe zu halbieren
- Sättigung statt Abschneiden im Festkommaformat
- Automatischer Vergleich eines Testbench-Protokolls mit dem Bus-Trace
- Generierung einer VHDL-Testbench mit den erwarteten Werten
- Übernahme der symbolischen Registernamen und Labels in `simpleCPU`

Anhang: Dateiverzeichnis mit Herkunft

Datei	Herkunft	Zeilen	Zweck
`src/Main.java`	gemischt	15	Einstiegspunkt, Menü oder Kommandozeile
`src/loader/Loader.java`	gemischt	265	Menü, Laden, Systemaufbau, Export-Steuerung
`src/CPU/CPU.java`	2018	8	CPU-Schnittstelle
`src/CPU/AbstractCPU.java`	gemischt	29	Basisklasse, Bit-Hilfsmethoden
`src/CPU/simpleCPU.java`	2018	139	Lern-CPU, Textinterpretation
`src/CPU/Cristian1604CPU.java`	2026	256	Emulation des cristian1604-MIPS
`src/CPU/SimulationException.java`	2026	8	Laufzeitfehler
`src/memory/Register.java`	gemischt	112	Registerfeld (Bit-Arrays)
`src/memory/specialproposeReg.java`	2018	4	Platzhalter Spezialregister
`src/memory/DataMemory.java`	2026	64	Datenspeicher am Bus
`src/assembler/Assembler.java`	2026	373	Zwei-Pass-Assembler
`src/assembler/Isa.java`	2026	109	Befehlssatz, Registernamen, Disassembler
`src/assembler/Program.java`	2026	50	Assembliertes Programm
`src/assembler/AssembledInstruction.java`	2026	8	Ein Maschinenbefehl mit Herkunft
`src/assembler/AssemblerException.java`	2026	17	Gesammelte Assemblerfehler
`src/bus/Bus.java`	2026	78	Datenbus
`src/bus/BusDevice.java`	2026	32	Geräteschnittstelle
`src/bus/BusTransaction.java`	2026	19	Protokolleintrag
`src/bus/BusException.java`	2026	8	Busfehler
`src/bus/MemoryMap.java`	2026	21	Speicherbelegung

Datei	Herkunft	Zeilen	Zweck
`src/accelerator/ComplexMultiplier.java`	2026	173	Verhaltensmodell Beschleuniger
`src/export/HexExporter.java`	2026	153	Hex-Ausgabe und Export
`src/selftest/SelfTest.java`	2026	138	Selbsttest
`src/Programm/kleine.asm`	2018	26	Beispiel Lern-CPU
`src/Programm/test.asm`	2018	1	Beispiel Lern-CPU
`src/Programm/test2.asm`	2018	26	Beispiel Lern-CPU
`src/Programm/testit.ams`	2018	9	Beispiel Lern-CPU
`src/Programm/complex_mul_acc.asm`	2026	40	Komplexe Multiplikation mit Beschleuniger
`src/Programm/complex_mul_sw.asm`	2026	77	Komplexe Multiplikation in Software
`src/Programm/cristian_beispiel.asm`	2026	16	Beispiel aus dem cristian1604-README
`README.md`	gemischt	–	Einleitung 2018, Abschnitt zur Erweiterung 2026
`Doku/`	2026	–	Diese Dokumentation (HTML und PDF)

Quellen

- C. Bottazzi: *MIPS – VHDL implementation of a Single-Cycle MIPS Processor*, github.com/cristian1604/mips, GPL-3.0
- D. M. Harris, S. L. Harris: *Digital Design and Computer Architecture*, Elsevier, 2007
- D. A. Patterson, J. L. Hennessy: *Computer Organization and Design*, Morgan Kaufmann, 2012
- MARS – MIPS Assembler and Runtime Simulator, Missouri State University